TECHNICAL DOCUMENT

Zoventro Platform
Technical Architecture & Integrations

A corporate and event engagement gaming platform: organizers book team activities, participants join from an invitation link and play live multiplayer games (Mystery Quest, Cook & Create), and results are generated automatically.

1. Platform overview
2. System architecture
3. Technology stack
4. Application components
5. Third-party integrations
6. Real-time engine (Socket.IO)
7. Key business flows
8. Data model & data retention
9. Security & data protection
10. API reference summary
11. Deployment & operations
12. Configuration reference

PRODUCT
Zoventro – Team Engagement Games

DOCUMENT VERSION
1.0

DATE
10 November 2026

CLASSIFICATION
Client – Technical

1. Platform overview

Zoventro lets HR and event teams run structured, timed team games for their employees. The platform has three user roles, each with its own interface:

Super Admin / Sub-admin

Manages packages, activities and games, game content, CMS pages, organizers, payments, email templates, site settings and roles (Laravel admin panel).

Organizer (HR)

Registers with email OTP, books an activity package, pays (Razorpay or offline), shares the invitation link, watches live event stats and downloads results and GST invoices.

Participant

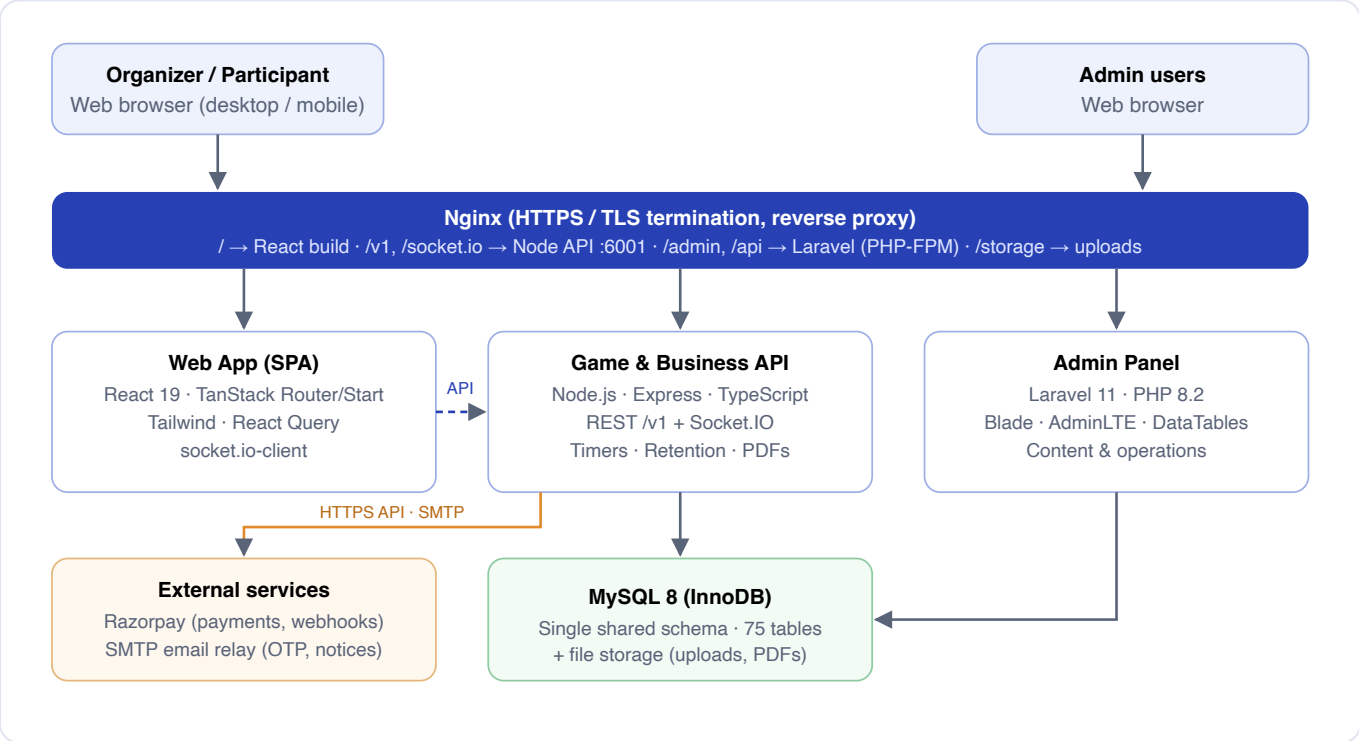
Opens the invitation link, verifies their email with an OTP, is auto-assigned to a group, waits in the lobby and plays the game in real time. No account or password is needed.

Games supported

- **Mystery Quest**: a role-based investigation game (investigator, suspects, witnesses) with case summary, questioning rounds, clue unlocks, a lie detector, witness pass-cards, accusation and verdict scoring.
- **Cook & Create**: a three-round collaborative cooking game: ingredient voting, recipe-step building with keep/remove votes, an impostor round with chat and "Double Down", then cross-group dish ratings and awards.

2. System architecture

The system is split into three applications that share one MySQL database, all behind a single Nginx web server on a Linux VPS.



- **Web App** is a static production build served by Nginx. It calls the Node API over HTTPS and keeps a WebSocket connection open during the lobby and the game.
- **Node API** owns all organizer, participant, payment and game logic, real-time events, game timers, PDF generation and automatic data retention.
- **Laravel Admin** is the back-office: master data, game content, CMS, settings and reporting. Values saved here (packages, settings, CMS pages, game durations) are read live by the Node API from the shared database.

3. Technology stack

Layer	Technology	Purpose
Frontend	React 19, TypeScript, TanStack Router / Start, Vite	Single-page web app for organizers and participants
UI	Tailwind CSS 4, Radix UI (shadcn), Lucide icons	Responsive, accessible components
State / data	TanStack React Query, Redux Toolkit, React Hook Form + Zod	Server caching, app state, form validation
Real-time	Socket.IO 4 (server + client)	Lobby, presence, live game events, live dashboard
Backend API	Node.js 20, Express 4, TypeScript	REST API `/v1/*`, game engine, payments
API libraries	mysql2, jsonwebtoken, express-validator, express-rate-limit, helmet, nodemailer, PDFKit, winston, moment	DB access, auth, validation, throttling, security headers, email, PDFs, logging
Admin panel	Laravel 11, PHP 8.2, Blade, AdminLTE, jQuery, Yajra DataTables	Back-office management
Admin libraries	barryvdh/laravel-dompdf, akaunting/setting	Admin-side invoice PDFs, key/value site settings
Database	MySQL 8 (InnoDB, utf8mb4)	Single shared relational store
Web server	Nginx + PHP-FPM	TLS, static hosting, reverse proxy
Process manager	PM2 (single instance, fork mode)	Keeps the Node API running and restarts it on failure
Hosting	Linux VPS (Ubuntu), e.g. AWS EC2 or Hostinger VPS	A persistent server is needed for WebSockets

4. Application components

4.1 Web App: main screens

Area	Screens
Public	Home with packages and pricing, CMS policy pages (`/policy/:slug`), privacy, terms
Organizer	Login / registration (email OTP), booking wizard (activity → package → schedule → billing → payment), dashboard with live stats and reschedule, groups, participants, payments and invoices, HR results, notifications, profile
Participant	Join via link (`/join/:linkToken`) with name, email and OTP, lobby with group status, Mystery Quest game, Cook & Create lobby / game / summary, results

4.2 Node API: modules

Module	Responsibility
Organizer	Registration, OTP login, JWT session, profile and billing (GST), bookings, reschedule (one-time), dashboard, event stats, notifications, account deactivation

Module	Responsibility
Payments	Razorpay order creation, checkout verification, failure recording, status recovery, webhook processing, GST invoice PDFs
Participant	Invitation link validation, join + OTP, group auto-assignment, package join limits, lobby and game summary
Game engine (Mystery Quest)	Game state, questions and answers, clue unlocks, lie detector, witness pass-cards, accusation, verdict scoring
Cook & Create	Round 1 voting, Round 2 steps and dish name, Round 3 chat, impostor vote and Double Down, ratings and awards
Results	Per-group results, organizer results tab, Mystery Quest and Cook & Create result PDFs (available for 1 hour after declaration)
Background services	Timer loop (every 5 s) that advances game phases; retention sweep (every 60 s) that auto-deletes joining data
Public	Packages, games, CMS pages, public site settings, payment methods, activity music

4.3 Laravel Admin: modules

Catalogue
Activities, activity games, packages (price, GST, group size, join limits), music

Game content
Mystery Quest story, roles, clues, scoring rules; Cook & Create templates, ingredients, rules, stage durations

Customers
Organizers, bookings, payments (mark as paid), invoices

Content
CMS pages (Privacy, Terms, Refund, Legal), blogs, FAQs, email templates

Settings
General, social media, OTP expiry, payment mode, logos and favicon

Access & ops
Roles and menu permissions, sub-admins, API logs, database export

5. Third-party integrations

5.1 Razorpay payment gateway

Item	Detail
Integration type	Razorpay Standard Checkout (browser) + Orders REST API (`https://api.razorpay.com/v1`) + Webhooks
Payment methods	Razorpay (cards, UPI, net-banking, wallets) and Offline / COD (activated by the organizer, then marked paid by the admin)
Order creation	The server calculates the amount from the package price + GST. The client never sends an amount.
Verification	After checkout, the server checks the HMAC-SHA256 signature of `order_id|payment_id` and re-fetches the payment from Razorpay before activating the booking.
Webhook endpoint	`POST /v1/webhooks/razorpay` : signature checked on the raw request body with a separate webhook secret. Every event is stored for audit and duplicate protection.

Item	Detail
Webhook events	`payment.captured`, `order.paid`, `payment.failed`, `refund.created`, `refund.processed`, `refund.failed`
Recovery	If the browser closes mid-payment, the webhook settles the booking, and `GET /v1/organizer/payment/status/:order_id` lets the app resume.
Test / Live	Switched by the `RAZORPAY_MODE` environment setting (test or live key pair). No code change is needed.

> Razorpay Dashboard setup: add the webhook URL `https://<your-domain>/v1/webhooks/razorpay`, select the six events above, and copy the webhook secret into the server environment.

5.2 Email (SMTP)

- Sent through **Nodemailer** over SMTP. The default relay is Brevo (Sendinblue), and any SMTP provider (Amazon SES, Gmail Workspace, Zoho and others) works through environment settings.
- Emails sent: organizer registration / login OTP, participant join OTP, booking and payment confirmation, notifications. Templates are editable in Admin → Email Templates.
- OTPs are 6 digits. Expiry is set by the admin (1–10 minutes, default 5), with a 60-second resend cooldown.

5.3 PDF generation

Document	Engine and rules
GST tax invoice	PDFKit (Node), from the organizer portal. Includes SAC code, GSTIN, and CGST/SGST or IGST from the supplier and customer state.
Mystery Quest results	PDFKit. Shown only after every group has finished, and downloadable for 1 hour after the result is declared.
Cook & Create awards	PDFKit. Same 1-hour window for the organizer (HR).
Admin invoice view	DomPDF (Laravel) from the admin organizer page

5.4 Internal integration: Admin ↔ API

The admin panel and the Node API share one MySQL schema. They do not call each other over HTTP. Settings, packages, game content and CMS pages saved in the admin take effect in the API on the next read. Uploaded images are stored in Laravel storage and served by Nginx under `/storage`, which the web app uses as its image base URL.

6. Real-time engine (Socket.IO)

The browser connects to `wss://<domain>/socket.io`. The server groups connections into rooms and pushes events as soon as game state changes.

Room	Joined by	Typical events
`group_<id>`	Participants of a group (lobby + game)	`lobby_updated`, `presence_updated`, `phase_changed`, `new_question`, `new_answer`, `clues_unlocked`, `lie_detector_started` / `_ended`, `new_vote`, `accusation_submitted`, `scores_updated`, `participant_left`
`cc-instance-<id>`	Cook & Create players	Round updates, `cc_dish_name_submitted`, `cc_double_down_offer`

Room	Joined by	Typical events
`organizer_<booking>`	Organizer dashboard	`event_stats_updated`, `organizer_notification`

- **Presence:** online, offline and left are tracked per participant. A 60-second grace period keeps a page refresh or short network drop from counting as leaving.
- **Timers:** phase timers are stored in the database and advanced by the server timer loop, so every player sees the same phase even after a refresh.
- **Scaling note:** rooms and timers run inside one Node process. Run a single PM2 instance, or add a Redis adapter before scaling to several instances.

7. Key business flows

7.1 Organizer booking & payment

Register / Login (email OTP) → Choose activity & package → Pick date & time (within booking window) → Billing & GST + consents → Razorpay checkout / COD → Signature + webhook verification → Booking active + invitation link

7.2 Participant join

Open invitation link → Name + email → Email OTP → Package join-limit check (verified users only) → Auto-assign to group → Lobby (group name, members, countdown)

7.3 Game session lifecycle

Lobby wait → Entry window closes → Full group → game starts · Incomplete group → "Session ENDED" → Timed phases / rounds → Verdict / awards → Group completed

- Groups that are not full when the entry window closes do not start. The lobby shows "Your group not complete participant." This is enforced on the server and in the app.
- If the key player (the investigator in Mystery Quest) leaves and does not return within 60 seconds, the group is closed as incomplete.

7.4 Results & auto-deletion

All played groups finished → Result declared → HR Results PDF available for 1 hour → Window expires → Joining data auto-deleted

8. Data model & data retention

The database has **75 tables**, created and versioned through Laravel migrations plus a few schema helpers run at API start-up. Main groups:

Domain	Main tables
Admin & access	users, roles, permissions, menus, settings, api_logs
Content	cms_pages, blogs (+ categories, tags, media), faqs, email_templates

Domain	Main tables
Catalogue	activities, activity_games, packages, activity_music, game_roles, game story / clue tables
Organizers & billing	organizers, organizer_bookings, organizer_billings, payments, payment_webhook_events, organizer_notifications
Live sessions	game_participants, game_groups, participant_sessions, timers, questions, answers, votes, lie_detector_rounds, group_accusations, witness_passcards, score_logs
Cook & Create	cc_game_templates, cc_ingredients, cc_game_instances, cc rounds / steps / votes / messages / ratings, cc_stage_durations

> **Data retention:** participant joining data (participants, groups, sessions and in-game content) is **permanently deleted automatically** once the 1-hour results window ends. If no group was played, it is deleted 1 hour after the scheduled session ends. Organizer accounts, bookings, payments and GST invoices are kept for accounting.

9. Security & data protection

Control	Implementation
Organizer authentication	Passwordless email OTP. After verification the server issues a signed JWT (HS256, 24 h), and the account status is checked on every request.
Participant authentication	Email OTP, then a random 64-character join token sent on every game request. The server works out identity from the token, not from IDs the browser sends.
Admin authentication	Laravel session login with role-based menu permissions and CSRF protection on all forms
Brute-force protection	Rate limits on OTP send and verify, plus a resend cooldown
Payment security	Server-side amount calculation, HMAC signature checks, constant-time comparison, webhook duplicate protection. No card data touches Zoventro servers (handled by Razorpay, PCI-DSS).
Transport & headers	HTTPS through Nginx, Helmet security headers, CORS allow-list
Database access	Parameterised queries throughout (protection against SQL injection)
Secrets	Kept in server environment files, never in the browser build. The public settings endpoint returns only an allow-listed set of keys.
Privacy	Minimal participant data (name and email only), automatic deletion after the event, consent captured at booking

> A separate pre-go-live security review is maintained by the development team. Its items must be closed before the production launch.

10. API reference summary

Base URL `https://<domain>/v1`. JSON request and response. Every response has the shape `{ success, message, data, errors }`.

Group	Auth	Main endpoints
/organizer	JWT (after OTP)	`register`, `verify-otp`, `login`, `verify-login`, `resend-otp`, `dashboard`, `profile`, `profile/billing`, `create-booking`, `complete-booking`, `update-session`, `event-stats/:id`, `notifications/:id`, `results`, `invoices`, `account/delete`
/organizer/payment	JWT	`verify`, `failed`, `status/:order_id`
/participant	OTP → join token	`verify-link/:token`, `join`, `resend-otp`, `verify-otp`, `lobby/:group`, `game-summary/:group`
/game	Join token	`state/:group`, questions, answers, lie detector, votes, accusation, witness pass-card
/cookandcreate	Join token	`state/:group`, `round1/*`, `round2/*`, `round3/*`, `rate`, `:group/awards`
/results	Join token / JWT	`:group`, `:group/pdf`
/public	None	`packages`, `games`, `cms`, `cms/:slug`, `settings`, `payment-methods`, music
/webhooks	Razorpay signature	`razorpay`

11. Deployment & operations

11.1 Server layout

Path / service	Role
`/var/www/p/frontend/dist`	Web app build served at `/`
`/var/www/p/apis` → PM2 `zoventro-api`	Node API on `127.0.0.1:6001`, proxied at `/v1` and `/socket.io` (WebSocket upgrade enabled)
`/var/www/p/public` → PHP-FPM	Laravel admin at `/admin` and legacy `/api`
`/var/www/p/storage/app/public`	Uploaded images and generated PDFs, served at `/storage`
MySQL 8	Database `zoventro` with a dedicated, least-privilege application user

11.2 Release procedure

Component	Commands
Code	`git pull origin main`
Admin (Laravel)	`composer install --no-dev -o` · `php artisan migrate --force` · `php artisan optimize`
API (Node)	`cd apis && npm ci && npm run build && pm2 restart zoventro-api`
Web App	`cd frontend && npm ci && npm run build` (output in `dist/`)

11.3 Monitoring & maintenance

- API logs through Winston and `pm2 logs zoventro-api`. Laravel logs in `storage/logs`. API request logs can be viewed in Admin → API Logs.
- Daily MySQL backup (`mysqldump`) and a backup of `storage/app/public`. Admin → Database Export gives an on-demand dump.

- TLS certificate through Let's Encrypt (certbot) with auto-renewal.
- Minimum recommended server: 2 vCPU, 4 GB RAM, 40 GB SSD, Ubuntu 22.04/24.04 LTS.

12. Configuration reference

Environment variable **names** only. Values are kept on the server and are never shared in documents.

Application	Variables
Node API (`apis/.env`)	`PORT` , `NODE_ENV` , `DB_HOST` , `DB_PORT` , `DB_USER` , `DB_PASSWORD` , `DB_NAME` , `JWT_SECRET` , `CORS_ORIGINS` , `MAIL_HOST` , `MAIL_PORT` , `MAIL_USERNAME` , `MAIL_PASSWORD` , `MAIL_FROM_ADDRESS` , `MAIL_FROM_NAME` , `RAZORPAY_MODE` , `RAZORPAY_TEST_KEY` / `_SECRET` , `RAZORPAY_LIVE_KEY` / `_SECRET` , `RAZORPAY_WEBHOOK_SECRET`
Admin (`.env`)	`APP_ENV` , `APP_DEBUG` , `APP_URL` , `APP_KEY` , `DB_*` , `MAIL_*` , `SESSION_SECURE_COOKIE`
Web App (build time)	`VITE_API_BASE_URL` , `VITE_STORAGE_BASE_URL` , `VITE_APP_NAME`